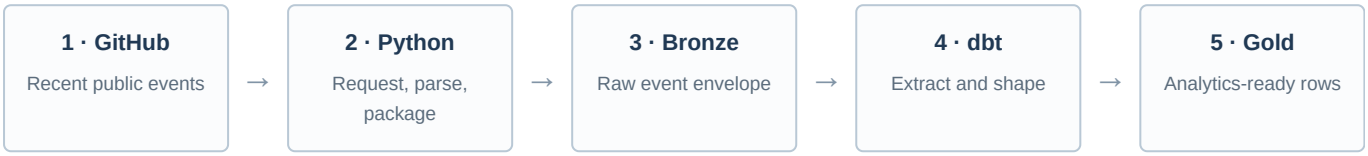
GitHub Telemetry Lakehouse

How one public GitHub event travels from an API response into BigQuery, through dbt, and into an analytics table — explained one brick at a time.

This project is a small data pipeline. It asks GitHub for recent public activity, reshapes each event into a row, saves those rows in BigQuery, and prepares easier-to-query tables with dbt.



First, the vocabulary

Data source	The system that produces or provides the information. Here, it is GitHub's public Events API.
Event	A record describing an activity, such as a push, a repository star, or a fork. It is not the Git repository itself.
API	A defined way for one program to request information or actions from another program.
JSON	A text format made of named fields, nested objects, and lists. It can represent data with different shapes.
Database	A managed place to store and retrieve data. BigQuery is the database/analytics service used here.
Pipeline	A sequence of repeatable steps that moves and prepares data.
ETL / ELT	ETL transforms before loading; ELT loads first, then transforms in the destination. This project follows ELT.

> **How to read the guide:** follow one example event through each numbered stage. At each stage, ask: what shape is the data, where is it, and what code changes it?

The guide describes the checked-in project code. It does not assume access to GitHub's private internal storage or event-processing systems.

What is stored at the source?

GitHub stores repositories, commits, files, and related information as part of its Git hosting service. This pipeline does not connect to that underlying storage. It requests a separate product interface: the public Events API at **/events**.

A repository

A project hosted on GitHub. Git stores version history as commits and file data. The pipeline does not download this history.

An event

A notification-like record that describes a public action associated with a repository. For example: “a push happened.” The event can include a summary of related details.

Important distinction

The API response is not a dump of GitHub’s database or a permanent archive. It is a recent feed of public events. This project requests that feed when it runs and saves the returned items. If an event has disappeared from the feed before a run, this code has no way to retrieve it from history.

Action happens
Example: repository receives a push

→

GitHub exposes an event
Event fields plus action-specific payload

→

Our program polls
GET request to /events

What does one event mean?

An event is a description of an action at a point in time. A `PushEvent` means a push occurred; a `WatchEvent` commonly describes a user starring a repository. The event’s `type` tells us which kind it is. The event’s `payload` contains details that depend on that type.

Field idea	Example meaning	Why it is useful
Event ID	Identifier for this event record	Recognize repeated copies of the same returned event
Type	`PushEvent`, `ForkEvent`, etc.	Group or filter actions
Actor	Account associated with the action	Identify the public account shown in the event
Repository	Repository associated with the action	Understand which project the event concerns
Payload	Type-specific details	Preserve the richer event details for later analysis

> **Scope check:** public event activity is not a complete measure of developer work. Private activity may not be visible, and an event feed is not a guaranteed full-history export.

How nesting works

JSON can place an object inside another object. This represents a parent-child relationship. A field can contain a simple value, an object with more named fields, or a list of values/objects.

```
{
  "id": "987654321",
  "type": "PushEvent",
  "created_at": "2026-11-05T09:40:00Z",
  "actor": { "id": 12345, "login": "sample-user" },
  "repo": { "id": 67890, "name": "sample-user/demo" },
  "payload": {
    "push_id": 456,
    "commits": [
      { "sha": "abc123...", "message": "Add report" },
      { "sha": "def456...", "message": "Fix query" }
    ]
  }
}
```

Read a path

```
actor → login
repo → name
payload → commits → item 0 → sha
```

Dots in a SQL JSON path express nested traversal, such as `$.actor.login`.

Why the payload varies

A push may include commit details. A star event may include an action. A fork can include fork information. The same payload columns do not make sense for every event type.

Object, array, and value

JSON shape	Example	Meaning
Object	`{"login":"sample-user"}`	Named fields grouped together
Array	`[{"sha":"abc"},{"sha":"def"}]`	Ordered list of items; indexes begin at 0 in many programming contexts
String	`"PushEvent"`	Text enclosed in quotation marks
Number	`12345`	Numeric value, not quoted

Why keep the original nested shape?

If we try to turn every possible payload field into a fixed column at the moment of download, different event types and future field changes make ingestion fragile. This project instead stores the full event as JSON text in Bronze, then extracts a selected set of fields later.

> **Think of it like this:** the whole JSON object is a parcel. A path such as `actor.login` is the address for one item inside the parcel. The project stores the parcel first and opens selected parts during modeling.

How Python gets the data

The function `fetch_github_events()` in `ingestion/fetch_and_push_bq.py` makes an HTTPS request and converts the response JSON into Python objects.

1. **Prepare headers.** The `Accept` header asks for GitHub's JSON representation. `User-Agent` identifies the client program.
2. **Optionally add a token.** If the environment variable `GITHUB_TOKEN` exists, the code sends it as a Bearer authorization header. Public events can be requested without it.
3. **Send GET.** `requests.get(..., timeout=15)` asks the server for the endpoint. HTTPS encrypts the network connection. The timeout limits how long this call waits.
4. **Check the status.** A successful response is parsed. For a non-success status, `raise_for_status()` stops the run with an error instead of pretending data arrived.
5. **Decode JSON.** `response.json()` turns JSON text from the response into a Python list containing dictionaries.

```
response = requests.get(GITHUB_API_URL,
                        headers=headers,
                        timeout=15)
response.raise_for_status()
events = response.json()
```

HTTP request

A message sent from the client (our Python program) to a server. GET means "return the resource." The response includes a status code, headers, and a body.

Deserialization

Turning JSON text into active Python data structures. After parsing, the program can use expressions such as `event.get("id")`.

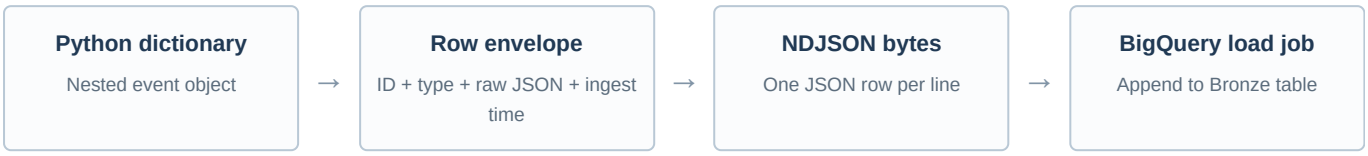
What is in memory now?

At this point, `events` is a Python list. Each item is a dictionary representing one event, and nested JSON objects become nested dictionaries/lists. This is temporary process memory; the code has not yet written the batch to a local data file or BigQuery.

> **Source limit:** the endpoint is a recent public feed, not a historical query API. Polling frequency and returned event volume affect what the pipeline sees. Do not infer complete coverage from a successful request.

From Python objects to BigQuery rows

The function `stream_to_bigquery(events)` keeps the returned batch in memory, wraps each event in a small database row, then submits the batch as a BigQuery load job.



1 · Build the row envelope

For each event, the code reads `id` and `type`, serializes the whole original event into `raw_payload`, and adds one `ingested_at` time shared by the batch. Events without an ID are skipped.

```
record = {
  "event_id": str(event.get("id")),
  "event_type": event.get("type", "Unknown"),
  "raw_payload": json.dumps(event),
  "ingested_at": ingested_at,
}
```

2 · Make newline-delimited JSON

NDJSON means each line is a separate, complete JSON object. BigQuery reads each line as one row. `StringIO` is a text buffer in RAM. The code rewinds it, obtains the string, encodes it to UTF-8 bytes, and wraps those bytes in `BytesIO` for the client library.

```
{"event_id":"987654321","event_type":"PushEvent","raw_payload":"{...}","ingested_at":"..."}
{"event_id":"987654322","event_type":"WatchEvent","raw_payload":"{...}","ingested_at":"..."}
```

3 · Load to the raw table

The BigQuery client submits a **load job** to `kd-bq-sandbox-2026.github_bronze.raw_events`. The schema tells BigQuery the column names and types. `WRITE_APPEND` means add these rows to existing rows. `job.result()` waits for completion and raises an error if the job fails.

Bronze column	Type	What is saved
`event_id`	STRING	Event identifier as text
`event_type`	STRING	For example, PushEvent
`raw_payload`	STRING	The entire original event serialized as JSON text
`ingested_at`	TIMESTAMP	When this pipeline batch was prepared

> **Memory detail:** this avoids creating a local batch file, but the response, text buffer, and encoded bytes can coexist in RAM briefly. “No local file” does not mean “no memory used.”

What “Bronze” and “Silver” mean here

These are names for data preparation stages. **Bronze** keeps a close-to-source record. **Silver** selects, types, and cleans fields into a more convenient shape.

Bronze · raw table

A persistent BigQuery table. In this project it is partitioned by the date of `ingested_at`, and partitions are configured to expire after seven days.

Silver · view

A saved SQL definition, not a separate stored copy of all result rows. When queried, BigQuery runs the SQL against Bronze and returns a shaped result.

How the staging SQL shapes data

1. Read Bronze columns: event ID, type, JSON text, and ingest time.
2. Use JSON paths such as `$.actor.login` and `$.repo.name` to extract selected nested values.
3. Convert text to typed values: for example, actor ID to an integer and event time to a timestamp.
4. Use `ROW_NUMBER()` grouped by event ID, keeping the most recently ingested copy when an event appears more than once.
5. Replace missing login/repository names with `unknown`, and omit rows whose event timestamp could not be parsed.

```
JSON_VALUE(raw_payload, '$.actor.login') AS actor_login
SAFE_CAST(JSON_VALUE(raw_payload, '$.actor.id') AS INT64) AS actor_id
```

What is deduplication?

Deduplication means treating repeated copies of the same event as one logical event. The API may return an event again on a later poll, and a retried load can also append the same rows again. The Silver view chooses one row per `event_id` for consumers.

> **Important:** the Bronze table remains append-only in this code. Deduplication in a view does not remove duplicate raw rows; it changes what Silver presents.

Why keep the raw payload?

It preserves fields the current model does not use. If the model later needs a new field, it can be extracted from the retained JSON while the Bronze partition still exists.

How the Gold table is produced

The mart model reads Silver, keeps events from the most recent seven days, and computes columns intended for reporting. In this project the Gold model is materialized as a BigQuery table, so its rows are stored after the dbt run.

Gold field	Mechanism	Example use
`event_date`	Date part of the UTC event timestamp	Group by day; partition the table
`event_hour_utc`	Hour extracted in UTC	Compare activity by UTC hour
`amer_local_time`, etc.	Convert UTC timestamp to named time zones	Display a timestamp in a selected region
`is_*_core_hours`	Check whether local hour falls in configured range	Approximate activity during regional office hours
`is_midnight_oil_pushed`	Check a late-night hour using the offset in the event timestamp	Flag a time-of-day pattern for review

What is a partition? What is clustering?

Partition

A table is organized into date-based sections. A query limited to a date range can often read fewer sections. Bronze partitions use ingestion date; Gold partitions use event date.

Clustering

BigQuery organizes data within a table around chosen columns to help some filters run efficiently. Here the model clusters by event type and the late-night flag.

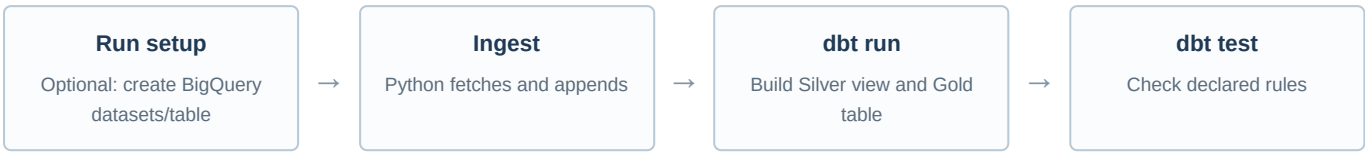
> **Interpretation limit:** UTC-to-New-York/London/Singapore conversion tells what the event time is in those time zones. It does not tell the actor's real location or work schedule. The late-night flag uses the timestamp's offset; when that timestamp is Z (UTC), the code checks UTC hours. It is a time-based heuristic, not evidence of burnout.

dbt tests

dbt is a tool for organizing and running SQL transformations. The project's YAML files declare checks such as "event ID is not null" and "event ID is unique." A test is a query that looks for rows violating a rule. It does not automatically correct bad data.

Who starts each step?

The same basic sequence can be started manually or by a scheduler. A scheduler starts a run at a chosen time; it does not change the meaning of the data steps.



Starter	What it does in this repo
`just`	Short command names for setup, ingestion, dbt, and the full sequence.
`scripts/run_pipeline.py`	Runs ingestion, then dbt run, then dbt test as subprocesses; stops if a command fails.
GitHub Actions	Runs from a hosted GitHub runner on a schedule, on push to main, or manually. It authenticates to Google Cloud before running.
Airflow	Defines a 15-minute DAG with ingestion → dbt run → dbt test. Its current Compose paths need adjustment to match the container mounts.

Credentials: what needs access?

- **GitHub API:** public events can be fetched without a token; `GITHUB_TOKEN` is optional.
- **BigQuery:** a Google identity is required to create query/load jobs and access the project datasets. Local runs use Application Default Credentials (ADC); CI currently expects a `GCP_SA_KEY` secret.
- **Safer CI setup:** replace a long-lived service-account key with GitHub OIDC and Google Workload Identity Federation where configured.

A few practical limits to remember

Coverage	Recent feed polling can miss events; it is not a complete archive.
Duplicates	Raw appends may repeat events; Silver deduplicates query results by ID.
Cost	Free-tier quotas and limits apply; “zero cost” depends on remaining within them.
Personal meaning	Public account activity and event times do not prove an individual’s location, work hours, or wellbeing.

> **One-sentence recap:** Python retrieves recent event JSON; Bronze stores each whole event inside a row; dbt extracts selected nested fields and builds a reporting table; tests check whether the modeled data follows declared rules.